



Apa itu NLP

- Natural Language Processing: cabang AI agar komputer memahami & memproses bahasa manusia
- Bukan sekadar string manipulation
- Input: teks atau suara (speech); lintas bahasa (EN, ID, Mandarin)
- Contoh: Google Translate, Siri, ChatGPT, search engine

► memahami ruang lingkup NLP

Tokenization

- Token = unit terkecil (kata, sub-kata, atau karakter)
- Contoh: 'Saya belajar NLP' -> 3 token
- Bahasa Indonesia lebih sulit karena imbuhan kaya
- NLTK pakai word_tokenize(); spaCy cukup nlp(text)

```
from nltk import word_tokenize; word_tokenize(text)
```

► langkah pertama preprocessing teks

Stopword Removal

- Hapus kata umum tak bermakna (yang, di, dari, dan)
- Fokus pada kata bermakna agar model lebih akurat
- EN: NLTK stopwords.words('english')
- ID praktiknya pakai nlp-id StopWord().get_stopword(); NLTK 'indonesian' alternatif valid

```
from nlp_id.stopword import StopWord; StopWord().get_stopword()
```

► membersihkan noise sebelum analisis/klasifikasi

Lemmatization vs Stemming

- Lemmatization: kata -> bentuk dasar kamus (menjalankan -> jalan)
- Lebih akurat dari stemming karena hasilnya kata valid
- ID: nlp-id Lemmatizer().lemmatize(); Sastrawi hanya rujukan historis
- Tantangan ID: imbuhan me-, -kan, -i, pe-, -an, ber-

```
from nlp_id.lemmatizer import Lemmatizer; Lemmatizer().lemmatize("saya menjalankan") # -> "saya jalan" (string, lowercase)
```

► normalisasi kata berimbuhan ke bentuk dasar

POS Tagging

- POS = Part of Speech: menandai kelas kata tiap token
- spaCy (EN) -> tag set Universal Dependencies: PRON, VERB, NOUN, ADP
- nlp-id (ID) -> tagset Indonesia (mirip Penn Treebank): NNP, VB, NN, IN, ADV (beda dari label UD ilustratif di slide)
- Penting untuk machine translation, QA, information extraction

```
from nlp_id.postag import PosTag; PosTag().get_pos_tag(text)
```

► analisis struktur gramatikal kalimat

Named Entity Recognition (NER)

- Identifikasi entitas: PERSON (orang), ORG (organisasi), LOC (lokasi)
- Contoh: 'Budi Santoso' -> PERSON, 'Navasena' -> ORG, 'Jakarta' -> LOC
- spaCy paling umum untuk NER bahasa Inggris
- Output dipakai untuk ekstraksi informasi terstruktur

```
for ent in nlp(text).ents: print(ent.text, ent.label_)
```

► ekstrak nama orang/tempat/organisasi dari teks

Sentiment Analysis

- Deteksi emosi/opini: skala negatif <-> positif
- TextBlob (EN): polarity in [-1.0, 1.0], subjectivity in [0.0, 1.0]
- Jebakan: polarity 0.0 pada teks Indonesia berarti 'tidak dikenali', bukan 'netral'
- ID: pakai IndoBERT via pipeline; leksikon hanya untuk teks Inggris

```
from textblob import TextBlob; TextBlob(text).sentiment.polarity
```

► mengukur opini/emosi pada review atau komentar

Klasifikasi Teks (Naive Bayes)

- Bag-of-words: dokumen jadi vektor contains(kata)=True/False, urutan diabaikan
- Akibatnya 'not good' dan 'good' bersinyal sama
- Naive Bayes murah dilatih dan keputusannya bisa dibaca manusia
- Dataset demo: movie_reviews (NLTK), 2.000 ulasan berlabel pos/neg

```
from nltk import NaiveBayesClassifier; NaiveBayesClassifier.train(set_train)
```

► kategorisasi dokumen dengan model yang bisa dijelaskan

Kebocoran Data & Baseline

- Seleksi fitur, vectorizer, scaler: semuanya di-fit pada data train saja
- Menghitung kosakata fitur dari seluruh korpus = data test ikut menentukan fitur
- Kesalahan ini tidak memunculkan error, hanya akurasi yang terlalu bagus
- Laporkan akurasi bersama baseline kelas mayoritas — tanpa itu angkanya tak berarti

```
kata_train = (w.lower() for dok, _ in dok_train for w in dok) # train saja
```

► agar angka evaluasi bisa dipercaya

Tantangan EN vs ID

- EN: morfologi sederhana, word order tetap (SVO), imbuhan terbatas (-s, -ed, -ing)
- ID: imbuhan kaya (jalan -> menjalani -> dijalankan), banyak loanword serapan
- Library beda per bahasa
- EN: nltk + spaCy; ID: nlp-id (Sastrawi rilis terakhir 2016, jangan dipilih untuk proyek baru)

► memilih tooling sesuai bahasa target

Dari Kata ke Vektor

- TF-IDF: vektor hitungan kata — buta sinonim ("suka" != "gemar")
- Embedding: vektor makna; sinonim -> vektor berdekatan
- cosine_similarity(emb_query, emb_dokumen) -> semantic search
- Inti RAG di Modul 05 (di sana ditambah FAISS)

► Fondasi retrieval & RAG

Subword & IndoBERT

- LLM memakai subword (BPE/WordPiece): "mempelajari" dipecah jadi potongan
- Kalimat Indonesia lebih banyak token di tokenizer English-centric
- IndoBERT via pipeline("sentiment-analysis") — tahan bahasa gaul
- Ekosistem ID: IndoNLU, IndoLEM, NusaBERT

► Jembatan langsung ke Modul 04 (LLM)

RAPIDS untuk NLP

- cuDF = pandas di GPU; cuML = scikit-learn di GPU
- nvtext (dalam cudf): tokenize, token_count, ngrams_tokenize, minhash
- Pre-installed di Colab runtime GPU — tinggal import cudf, cuml
- MinHash + groupby = dedup near-duplicate ala NeMo Curator

```
s = cudf.Series(teks); s.str.tokenize(); s.str.ngrams_tokenize(2, separator="-")
```

► preprocessing teks skala besar (100 ribu+ dokumen)

Zero Code Change (GPU)

- cudf.pandas: kode pandas lama lari di GPU tanpa diubah
- cuml.accel: estimator sklearn dialihkan ke cuML
- Operasi tak didukung → fallback otomatis ke CPU (tidak crash)
- Data kecil bisa lebih lambat — GPU menang di skala
- Warm-up wajib: jangan pernah mengukur pemanggilan CUDA pertama

```
python -m cudf.pandas -m cuml.accel pipeline.py
```

► migrasi cepat pipeline pandas/sklearn ke GPU

Quick-Ref Library / Tool NLP

Library	Bahasa	Fungsi utama	API contoh
NLTK	EN + ID	Tokenize, stopwords, klasifikasi	word_tokenize(), NaiveBayesClassifier
spaCy	EN	NER, POS, dependency parsing	nlp(text), token.pos_, doc.ents
TextBlob	EN	Sentiment (polarity, subjectivity)	TextBlob(text).sentiment
Sastrawi	ID	Stemming ID — rilis terakhir 2016, rujukan historis	StemmerFactory().create_stemmer()
nlp-id	ID	Tokenizer, POS tag, lemmatizer, stopwords	PosTag().get_pos_tag(), StopWord().get_stopword()
IndoBERT	ID	Sentiment / klasifikasi (model)	transformers pipeline
wordcloud	EN + ID	Visualisasi word frequency	WordCloud().generate(text)

Istilah Kunci

- Token** — Unit terkecil teks hasil tokenization (kata, sub-kata, atau karakter).
- Stopword** — Kata umum tak bermakna yang diabaikan (EN: the, is; ID: yang, di, dari, dan).
- Lemmatization** — Mengubah kata ke bentuk dasar kamus yang valid (menjalankan -> jalan).
- Stemming** — Memotong imbuhan ke akar kata, hasil bisa bukan kata kamus; Sastrawi untuk ID.
- POS (Part of Speech)** — Kelas kata tiap token; spaCy pakai tag UD (NOUN, VERB), nlp-id pakai Penn-Treebank ID (NN, VB, NNP).
- NER** — Named Entity Recognition: deteksi entitas PERSON, ORG, LOC dalam teks.
- Polarity** — Skor sentimen TextBlob dari -1.0 (negatif) sampai +1.0 (positif).
- Naive Bayes** — Algoritma probabilistik untuk klasifikasi teks; fitur informatifnya bisa dibaca manusia.
- Embedding** — Vektor angka yang merepresentasikan makna teks.
- Cosine similarity** — Ukuran kedekatan dua vektor (1 = searah/semakna).
- Subword** — Potongan kata yang menjadi satuan token LLM (BPE/WordPiece).
- TF-IDF** — Bobot kata berdasarkan frekuensi di dokumen vs keseluruhan korpus.
- Kebocoran data** — Informasi dari data test ikut membentuk model atau fiturnya; akibatnya akurasi terlalu optimistik.
- Baseline kelas mayoritas** — Akurasi kalau selalu menebak kelas terbanyak; pembandingan wajib untuk setiap skor akurasi.
- Warm-up** — Pemanggilan GPU pertama untuk menyerap biaya inisialisasi CUDA sebelum pengukuran waktu dimulai.